

- Maven Configuration Notes:
 - Do not type the version number of a spring boot dependency in the pom.xml file.
 - If you ctrl+click the spring-boot-starter-parent artifact id in the parent tag, it will take you to the pom file of the spring-boot-starter-parent
 - If you ctrl+click the the artifact id spring-boot-dependencies in the parent tag of the spring-boot-starter-parent pom file, it will take you the spring-boot-dependencies pom file
 - In the spring-boot-dependencies pom file it has all of the listed dependencies and versions that are compatible with each other. If you remove the versions, spring boot will use the versions in this parent file to select the right version
 - Lombok is a dependency that helps with using annotations that allow you create getter and setter methods easier
- The ResponseEntity class represents the http response that gets sent back to a client. Its properties include:
 - The status code that lets you know if the request is okay, or errored out
 - The header that contains meta-data, such as content type, security tokens, and cache controls
 - The body of the response, which is the actual content
- JpaSpecificationExecutor can be used for sending dynamic queries on the database for more complicated search capabilities
 - You add it to the list of classes you want to extend/inherit in your repository class
 - Spring actually has query methods
 - Examples:
 - findByFieldNameContaining(String queryName)
 - The method name gets converted into a sql query that uses parameter you passed to form the where condition on the field being referenced in the method
 - However, to combine filters, we would need to hard code each combination of criteria with a query method
 - However, the alternative for more complicated queries is using the JpaSpecificationExecutor
 - Create a new class that imports the specification class and handles the criteria building
 - Note: specification is a functional interface, which is an interface class with one abstract method
 - When you create a new specification object, you have to pass an implementation for the abstract method that will return a predicate(which is a function that returns a boolean value and is used to test a condition on an object)

- Note: functional interfaces can have as many static and default methods as it wants as long as it has one abstract method
- Predicate method will have three parameters
 - Root - represents the entity(class that's supposed to represent structure of data table)
 - `root.get(<entity-variable>)` is like selecting a field on a table. Can be used to grab a table field for setting a condition
 - Query - specifying the order, distinct, or group by clauses of a sql query
 - criteriaBuilder - basically the where clause of a sql query
 - `criteriaBuilder.equals(<field-name>,<provided-value>)` - basically creates a where condition for the sql query that checks if a provided value is equal to the value in the record that it's looking at
 - `criteriaBuilder.likes(<field-name>,<provided-value>)` - checks if provided value is contained within string value of field
 - Use
- Implementation could look like this:
 - Return new Specification<class-entity>() { @Override Public Predicate toPredicate(Root<class-entity> root, Query<class-entity> query, CriteriaBuilder<class-entity> criteriaBuilder) { Return criteriaBuilder.equals(root.get(<field-name>),<provide-value>);}}
 - I would copy this and paste this in notepad and add the necessary spaces to make this look cleaner and make more sense
- Or the implementation could be converted to a lamda expression
 - (root, query, criteriaBuilder) -> `criteriaBuilder.equals(root.get(<field-name>),<provide-value>);` – much cleaner, use this one
 - Functional interface implementation can allow you to convert expressions to lamda expressions
- Should also import the Root, Query, and criteriaBuilder classes for this
- Once it's added to the list of extended classes, you can use a method in the interface called `findAll` that takes in the specification object you provide it
- You can use a `PostMapping` method on the controller so you can send a full data object that will represent the search criteria
 - Pass the search criteria to a `findByCriteria` method that will be implemented by the Dao that will utilize the specification class created

- For the dataCriteria object being passed, set the value of the map to type object. To handle casting, use instanceof and assign it to a variable with the type it represents. For example:
 - Value(object) instanceof <variable-type> variableName
- In the findByCriteria method, you'll be able to initialize a specification object like this: Specification.unrestricted()
- After that, you can use a series of if-statements that check if a criteria field is empty, if it's not you can use a .or(predicate) or .and(predicate) concatenation method to add to the specification object
 - The parameter you're passing to the .or() and .and() methods are the Specification objects that you return in the Specification methods you created
 - Make sure you re-assign the spec variable the result of calling the .or() or .and() on the original spec. Just calling those methods on the spec won't change the object, but will return a new spec object with the or/and with the condition(what the specification object represents) appended.